An Unending Battle: A Thorough Analysis of the RISC vs CISC Debate and a Verilog Simulation of the RISC Architecture

*Not an IEEE research paper. Done as part of a Computer Architecture Project

Kabir Kanha Arora
Sophomore Year, Computer Science and Engineering
Shiv Nadar University
Greater Noida, India
ka682@snu.edu.in
1710110165

Komal Talwar
Sophomore Year, Computer Science and Engineering
Shiv Nadar University
Greater Noida, India
kt252@snu.edu.in
1710110183

***Abstract*—This paper revisits the unsettled battle for supremacy between RISC and CISC instruction sets, gives a thorough analysis and conclusion for the same, and finishes off with a verilog based simulation of the RISC microcontroller.**

***Index Terms*—isa, risc, cisc, arm, computer, architecture, verilog, simulation**

I. Introduction to ISA

Instruction Set Architecture (ISA) is also known as CPU architecture. ISA allows communication between software and hardware. It is also a group of commands for CPU and machine language. So basically, ISA tells you how your processor is going to process your program's instructions. An instruction, just to clarify, is also known as op-code or operation code. Some examples of op-codes are `ADD`, `JUMP`, `LOAD` and `STORE`.

Instruction set architecture serves as a bridge between the software and hardware layers of a computer system, binding the two by acting as a means of communication of the processor. It lists all the instructions that an operator can give. Among the components of an instruction set architecture, some of these are registers, addressing modes, data-types, memory management, devices and exception handling, power management and multi-threading support. The instructions that exist in the set are used to alter the machine state, data or are used for performing an I/O operation.

Most modern ISAs are classified into **RISC** (Reduced instruction set Computer) and **CISC** (Complex instruction set computer) on the basis of their architectural complexity. Other architectures include *VLIW, LIW and EPIC*. These are used to rely less on hardware and exploit instruction level parallelism. All these architectures vary in their complexity and the portion of the computer system that they are willing to exploit. A study conducted, which focused on the effect of ISAs on code density, observed that code density is mostly affected by the number of registers being used, length of an instruction, hardware divisors, number of operands, etc. The study conjointly showed that x86 (which has CISC in use) is one of the densest ISAs. As another observation by experimentation, it has been seen that compression of immediate values in instructions gives us a **30%** reduction in code size along with a significant **10%** reduction in processor's power consumption.

Several studies have concluded that RISC incorporates a performance edge over CISC processors and that CISC requires more aggressive microarchitecture optimizations to overcome the performance bottleneck. There exist other studies that say with time and rapid evolution in technology, the gap between RISC and CISC won't matter as *Moore's law* starts to diminish. Our study emphasizes on RISC and its simple differences with CISC.

II. The two types of ISA

A. Complex Instruction Set Computing (CISC)

There are two types of ISA. The first one is **CISC**, which stands for *Complex Instruction Set Computing*. It is an architecture with a large number of instruction sets. For instance, the microprocessor of CISC is Intel x64, which the 64-bit version of the x86 instruction set. They are mostly found in desktop and laptops.

B. Reduced Instruction Set Computing (RISC)

Now, on the other hand, **RISC**, yes with a C, stands for *Reduced Instruction Set Computing*, is another type of ISA which has a smaller set of instructions.

For eg: ARM processors are extensively used in consumer electronic devices such as tablets, smartphones and even gaming consoles. RISC is a form of microprocessor architecture that utilizes a small, highly-optimized set of instructions, instead of an additional specialised set of instructions often found in alternative types of architectures.

III. The Popularity of CISC

Up until about the 1980s, most computer processors (the CPUs) were getting more and more complicated and the idea was that if you could make the hardware sort of mirror the software, then of course it would be quicker. So as you wrote a program (which was probably exclusively in *C* or *Fortran* in those days), it produced certain instructions. If those instructions could be executed more and more in the hardware with less and less instructions, that would make the whole thing go much faster. And obviously, there were also other considerations like memory being expensive and so if you had shorter programs, such that each instruction could do more complicated things and take up less space, that would be better. The speed of memory and the complexity of processes themselves was different, and so was the number of transistors involved.

IV. The emergence of RISC

And then in the early 1980s, this idea came along of having a reduced complexity and that's what got it its name. Although, as we'll see later, it's not really about having a smaller number of instructions, it's more about how you use those instructions. Now, the argument was put forward like this by **David Patterson**, a renowned computer scientist and author:

> *"The simplicity of the instruction set and addressing modes allows most instructions to execute in a single machine cycle, and the simplicity of each instruction guarantees a short cycle time. In addition, such a machine should have a much shorter design time." [1]*

The original idea emerged from the observation that many features that had been incorporated in traditional CPU designs were not being used to facilitate coding and were just mere features occupying space. These complex features also took more than one clock cycle to be completed. One of the fundamental ideas behind RISC was that every instruction can be executed once per cycle, which was very hard with CISC, because each instruction was doing so many things, you'd have to wait two, three, four, five or maybe even more cycles for it to complete all the tasks.

Now in a RISC set-up, with every cycle you could actually execute an instruction. Performance gap had started to increase between the processor and main memory and so number of main memory accesses had to be limited. When CISC'S observed performance had started to degrade because of its complexity, other architectures' exploration had started. The idea was to first decrease the number of main memory accesses, as it was observed that due to frequent accesses to the memory, speed gets drastically affected.

Once you start thinking along these lines, you can make some other assumptions, such as all instructions having a fixed size in memory, which means it's easy to decode them and easy to send them down towards the pipeline into execution because you're dealing with the same fixed amount of data.

However, in the x86, the instructions can be of variable length (upto approx 15 bytes long) and one doesn't really know how many bytes are related to a particular instruction until one reads the entire thing.

What RISC architecture does is that it keeps its instruction set very simple. It has very few instructions where data is asked from the memory (typically Load and Store). This is another reason why RISC processors have a lot of registers. The register set is **homogeneous**, which provides extra flexibility. Addressing modes are limited and simple and so are data types. Load and store are the key elements of this architecture.

Another feature that makes RISC's performance better is **pipelining**. Pipelining increases the throughput by reducing the CPI and makes instructions be completed in one clock cycle. It is based on the concept of parallelism and idle states of components. Uniform instruction encoding allows faster decoding. Operands and opcode bits fixed in IR helps overcome the complexity of design.

V. RISC v/s CISC

The major difference between these two that most people never notice is that CISC starts with a C, and RISC starts with an R. *Got you there, eh?* Now that you're paying attention, let's move on to the real stuff.

To understand these two components, let us take an example of multiplying two variables: **A*B**. For the CISC, they have a specific instruction called `MULT`, also known as a complex instruction. Thus, the entire task of multiplying two numbers can be completed with one instruction. For the RISC, the `MULT` command is divided into **three** separate commands:

```
LOAD R1, A
LOAD R2, B
PROD A, B
STORE R3, A
```

- One of the most *fundamental differences* between RISC and CISC is that RISC does not do any operations directly on memory.
  For eg: If you have the number 7 in address 100 in memory and you want to add 1 to it, in x86 and other CISC kind of instruction sets, you can say add 1 to the number that is in address 100. Now, to do that, the CPU first of all has to go and fetch the content from address 100 and bring it into the CPU, then it has to add 1 to it and has to write it back out again in the same memory address. So we see that it does all that following just one machine code instruction that it read, but obviously it is doing three separate tasks.
  Now, with RISC, what you have to do, is that you actually have to have three instructions and the first instruction says go and fetch the memory contents into a register, the second instruction says add one to this register and the third instruction tells the computer to write the contents of the register back out into the memory. The reason for this is to facilitate the fact that one instruction is executed

every cycle, and the instructions must be small enough (reduced enough).

- Now this idea of having one instruction per cycle led to the idea of a pipeline, where the instruction starts to be processed, to be decoded, and the different internal states start to be modified as it goes down the pipeline, and then finally the instruction is said to have been executed. Now, the original RISC had a very short pipeline, maybe even two stages and because of that, they had to deal with the whole issue of *branching* i.e., what happens when the CPU jumps off somewhere. So they came up with the idea of what is called a ***delayed jump***, and what that means is that when the instruction is executed, the next instruction was already coming down the pipeline. So what they liked was that after a jump, you had the concept of a `NO OP` or a `NOP`:

DELAYED JUMP

```
200 LOAD X,A
201 ADD 1,A
202 JUMP 206
203 NOP
204 ADD A,B
205 SUB C,B
206 STORE A,Z
```

This `NOP` operation actually does nothing. It's there so that when you finally did branch somewhere else, that last instruction was already hot on the heels of the jump instruction, so it doesn't affect the outcome of the program. In fact , they said that you could actually optimise it in the sense that you could have the jump instruction first and because you knew the next instruction was coming up behind it, it could be the last instruction that you wanted to happen before the jump instruction, so you wrote `JUMP` first, then the last instruction, both of which would get executed and this is why it was called delayed jump.

OPTIMISED DELAYED JUMP

```
200 LOAD X,A
201 JUMP 205
202 ADD 1,A
203 ADD A,B
204 SUB C,B
205 STORE A,Z
```

This work in delayed jump led to one of the big differences between RISC and CISC.

It is a common misconception that CISC is always used with Von Neumann whereas RISC is used with Harvard architecture. However, there is no correlation between Instruction Set (RISC and CISC) with architecture of the processor (Harvard Architecture and Von Neumann Architecture). Both instruction set can be used with any of the architecture.

- An important difference is that CISC has more instruction sets than RISC.
- The instruction cycle in CISC is more complex than RISC, whereas RISC must combine simple operations from its reduced instruction set.
- CISC is mostly emphasized on hardware while RISC is mainly focused on software.
- The execution time for CISC has multiple number of cycles per instruction. For RISC, it is one cycle per instruction.
- CISC CPUs are much larger than those of RISC. This is because they have larger instruction libraries (and a fan) to cool it. Otherwise, it would blow up in your face *(haha!)*.
- Due to the reduced instruction set, RISC CPUs can be put onto a single chip. This makes them very light and allows them to make portable devices.
- Since CISC has a bigger CPU, it obviously has larger energy requirements, whereas RISC has lower energy requirements and can actually go into sleep mode when not in use.
- CISC CPUs take some time to process the instructions while, RISC CPUs are faster. However, it has been observed that RISC can only perform simple tasks quicker than CISC, but CISC is more capable of handling intensive tasks, thereby making it faster for a larger variety of tasks.

VI. Applications

RISC and CISC microprocessors find applications in various different fields.

RISC processors are used in video processing, telecommunication, image processing, etc. while CISC microprocessors find applications in fields where more power consumption is required.

RISC finds more applications in very small devices as it is more power efficient because the extra instruction decode logic in a CISC processor has a cost. Common RISC microprocessors include *Alpha, ARC, ARM, AVR, MIPS, PA-RISC, PIC, Power Architecture*, and *SPARC*. A few examples of CISC processors are the *System/360, VAX, PDP-11, Motorola 68000* family, AMD and Intel x86 CPUs.

Now, a lot of people make comments like, *"Yeah, but the 90s showed us that RISC lost and CISC won."* And they're probably talking about, say, servers or desktop PCs because they all use x86. Of course, that's nonsense for two main reasons. We all use RISC processors all the time - in our smartphones, in our Chromebooks, in our tablets, etc. Also, people forget that the world is not actually divided into just x86 and ARM. Take the example of the IBM powered Watson AI, which uses power architecture, which is RISC based. So RISC in servers, RISC in phones is pretty much here. The only CISC dominated place is actually desktop PCs, which leads people to believe that CISC is the winner, which is of course, far from the truth.

VII. Verilog Simulation

A. Overview

In our study to understand RISC processors, we tried to implement a 16-bit RISC processor and simulate basic operations on it. In our mission to do so, we carefully designed an instruction set with opcodes and instruction formats that the processor will be referring to for its various operations. Operations such as ADD, SUB, INVERT, Shift Left, etc. have been taken into consideration while building the processor's ALU and accordingly the entire processor's design has been laid out. The design has several components in it, mainly including an Instruction Memory, General Purpose Registers (GPRs), ALU Control Unit, ALU, Data Memory and several multiplexers to connect all of these and provide inputs for multiple operations. The flow of the simulation is in such a way that when instructions are specified in the test.prog file which is linked to the instruction memory, the connection has been made and the file is read, performing those instructions sequentially. A number of operations have been taken into consideration by us. The outputs obtained in registers are just as how we would predict it to be. As we have realised now that RISC has several General-purpose registers, these can be used to store results of various operations. A computer simulation allows the user to visualise the working procedure of a microprocessor and actually realise its working and mechanism. Talking about realisation, it is now possible for us to visualise the execution of instructions.

B. Instruction Format

OPCODE	INSTRUCTION
0000	Load
0001	Store
0010	Add
0011	Subtract
0100	One's Complement
0101	Logical Shift Left
0110	Logical Shift Right
0111	Bitwise And
1000	Bitwise Or
1001	Set on less than
1010	Hamming Distance (Xor)
1011	Branch on Equal
1100	Branch on Not Equal
1101	Jump

List of Opcodes

Op(4)	Rs1(3)	Ws(3)	Offset(6)

Memory Access: Load

Op(4)	Rs1(3)	Rs2(3)	Offset(6)

Memory Access: Store

Op(4)	Rs1(3)	Rs2(3)	Ws(3)	Useless(3)

Data Processing

Op(4)	Rs1(3)	Rs2(3)	Offset(6)

Branch

Op(4)	Offset(12)

Jump

C. Verilog Codes

C:/Users/Komal/Downloads/RISCprocessor-20190423T091317Z-001/RISC

```
`include "Parameter.v"
// Verilog code for RISC Processor
// Verilog code for Instruction Memory
module Instruction_Memory(
 input[15:0] pc,
 output[15:0] instruction
);

 reg [`col - 1:0] memory [`row_i - 1:0];
 wire [3 : 0] rom_addr = pc[4 : 1];
 initial
 begin
  $readmemb("C:/Users/Komal/Downloads/RISC
 end
 assign instruction =  memory[rom_addr];

endmodule
```

Verilog code for instruction memory

C:/Users/Komal/Downloads/RISCprocessor-20190423T091317Z-001/RISCprocessor/RISCproc

```
`timescale 1ns / 1ps
module GPRs(
 input    clk,
 // write port
 input    reg_write_en,
 input  [2:0] reg_write_dest,
 input  [15:0] reg_write_data,
 //read port 1
 input  [2:0] reg_read_addr_1,
 output  [15:0] reg_read_data_1,
 //read port 2
 input  [2:0] reg_read_addr_2,
 output  [15:0] reg_read_data_2
);
 reg [15:0] reg_array [7:0];
 integer i;
 // write port
 //reg [2:0] i;
 initial begin
  for(i=0;i<8;i=i+1)
   reg_array[i] <= 16'd0;
 end
 always @ (posedge clk ) begin
   if(reg_write_en) begin
    reg_array[reg_write_dest] <= reg_write_data;
   end
 end
 assign reg_read_data_1 = reg_array[reg_read_addr_1];
 assign reg_read_data_2 = reg_array[reg_read_addr_2];
endmodule
```

Verilog code for register file

C:/Users/Komal/Downloads/RISCprocessor-20190423T091317Z-001/RISCprocessor/RISCprocessor.srcs/sources

```verilog
`include "Parameter.v"
module Data_Memory(                    // Verilog code for RISC Proces
                                       // Verilog code for data Memory
 input clk,
 input [15:0]   mem_access_addr,   // address input, shared by read
 input [15:0]   mem_write_data,   // write port
 input     mem_write_en,
 input mem_read,
 // read port
 output [15:0]   mem_read_data
);
reg [`col - 1:0] memory [`row_d - 1:0];
integer f;
wire [2:0] ram_addr=mem_access_addr[2:0];
initial
 begin
  $readmemb("C:\Users\Komal\Downloads\RISCprocessor-20190423T091317Z-
  f = $fopen(`filename);
  $fmonitor(f, "time = %d\n", $time,
  "\tmemory[0] = %b\n", memory[0],
  "\tmemory[1] = %b\n", memory[1],
  "\tmemory[2] = %b\n", memory[2],
  "\tmemory[3] = %b\n", memory[3],
  "\tmemory[4] = %b\n", memory[4],
  "\tmemory[5] = %b\n", memory[5],
  "\tmemory[6] = %b\n", memory[6],
  "\tmemory[7] = %b\n", memory[7]);
  `simulation_time;
  $fclose(f);
 end
 always @(posedge clk) begin
  if (mem_write_en)
   memory[ram_addr] <= mem_write_data;
 end
 assign mem_read_data = (mem_read==1'b1) ? memory[ram_addr]: 16'd0;
endmodule
```

Verilog code for data memory

C:/Users/Komal/Downloads/RISCprocessor-20190423T091317Z-001/RISCpro

```verilog
`timescale 1ns / 1ps
// Verilog code for 16-bit RISC processor
// ALU_Control Verilog code
module alu_control( ALU_Cnt, ALUOp, Opcode);
 output reg[2:0] ALU_Cnt;
 input [1:0] ALUOp;
 input [3:0] Opcode;
 wire [5:0] ALUControlIn;
 assign ALUControlIn = {ALUOp,Opcode};
 always @(ALUControlIn)
 casex (ALUControlIn)
   6'b10xxxx: ALU_Cnt=3'b000;
   6'b01xxxx: ALU_Cnt=3'b001;
   6'b000010: ALU_Cnt=3'b000;
   6'b000011: ALU_Cnt=3'b001;
   6'b000100: ALU_Cnt=3'b010;
   6'b000101: ALU_Cnt=3'b011;
   6'b000110: ALU_Cnt=3'b100;
   6'b000111: ALU_Cnt=3'b101;
   6'b001000: ALU_Cnt=3'b110;
   6'b001001: ALU_Cnt=3'b111;
  default: ALU_Cnt=3'b000;
  endcase
endmodule
```

Verilog code for ALU Control Unit

C:/Users/Komal/Downloads/RISCprocessor-20190423T091317Z-001/RISCproc

```verilog
`timescale 1ns / 1ps
// Verilog code for RISC Processor
// Verilog code for ALU
module ALU(
 input  [15:0] a,  //src1
 input  [15:0] b,  //src2
 input  [2:0] alu_control, //function sel

 output reg [15:0] result,  //result
 output zero
    );

always @(*)
begin
 case(alu_control)
 3'b000: result = a + b; // add
 3'b001: result = a - b; // sub
 3'b010: result = ~a;
 3'b011: result = a<<b;
 3'b100: result = a>>b;
 3'b101: result = a & b; // and
 3'b110: result = a | b; // or
 3'b111: begin if (a<b) result = 16'd1;
    else result = 16'd0;
    end
 default:result = a + b; // add
 endcase
end
assign zero = (result==16'd0) ? 1'b1: 1'b0;

endmodule
```

Verilog code for ALU

C:/Users/kabir/Vivado/RISCprocessor/RISCprocessor.srcs/sources_1/new/Datapath_Unit.v

```verilog
`timescale 1ns / 1ps
module Datapath_Unit(
 input clk,
 input jump,beq,mem_read,mem_write,alu_src,reg_dst,mem_to_reg,reg_
 input[1:0] alu_op,
 output[3:0] opcode
);
 reg  [15:0] pc_current;
 wire [15:0] pc_next,pc2;
 wire [15:0] instr;
 wire [2:0] reg_write_dest;
 wire [15:0] reg_write_data;
 wire [2:0] reg_read_addr_1;
 wire [15:0] reg_read_data_1;
 wire [2:0] reg_read_addr_2;
 wire [15:0] reg_read_data_2;
 wire [15:0] ext_im,read_data2;
 wire [2:0] ALU_Control;
 wire [15:0] ALU_out;
 wire zero_flag;
 wire [15:0] PC_j, PC_beq, PC_2beq,PC_2bne,PC_bne;
 wire beq_control;
 wire [12:0] jump_shift;
 wire [15:0] mem_read_data;
  initial begin // PC
  pc_current <= 16'd0;
 end
 always @(posedge clk)
 begin
   pc_current <= pc_next;
 end
 assign pc2 = pc_current + 16'd2;
 Instruction_Memory im(.pc(pc_current),.instruction(instr)); // in
 assign jump_shift = {instr[11:0],1'b0};  // jump shift left 2
 assign reg_write_dest = (reg_dst==1'b1) ? instr[5:3] :instr[8:6];
 // register file
 assign reg_read_addr_1 = instr[11:9];
 assign reg_read_addr_2 = instr[8:6];
 // GENERAL PURPOSE REGISTERs
 GPRs reg_file
 (
  .clk(clk),
  .reg_write_en(reg_write),
  .reg_write_dest(reg_write_dest),
```

C:/Users/kabir/Vivado/RISCprocessor/RISCprocessor.srcs/sources_1/new/Datapath_Unit.v

```
 .reg_write_dest(reg_write_dest),
 .reg_write_data(reg_write_data),
 .reg_read_addr_1(reg_read_addr_1),
 .reg_read_data_1(reg_read_data_1),
 .reg_read_addr_2(reg_read_addr_2),
 .reg_read_data_2(reg_read_data_2)
);
// immediate extend
assign ext_im = {{10{instr[5]}},instr[5:0]};
// ALU control unit
alu_control ALU_Control_unit(.ALUOp(alu_op),.Opcode(instr[15:12]),.ALU
// multiplexer alu_src
assign read_data2 = (alu_src==1'b1) ? ext_im : reg_read_data_2;
// ALU
ALU alu_unit(.a(reg_read_data_1),.b(read_data2),.alu_control(ALU_Cont
// PC beq add
assign PC_beq = pc2 + {ext_im[14:0],1'b0};
assign PC_bne = pc2 + {ext_im[14:0],1'b0};
// beq control
assign beq_control = beq & zero_flag;
assign bne_control = bne & (~zero_flag);
// PC_beq
assign PC_2beq = (beq_control==1'b1) ? PC_beq : pc2;
// PC_bne
assign PC_2bne = (bne_control==1'b1) ? PC_bne : PC_2beq;
// PC_j
assign PC_j = {pc2[15:13],jump_shift};
// PC_next
assign pc_next = (jump == 1'b1) ? PC_j :  PC_2bne;
/// Data memory
 Data_Memory dm
  (
   .clk(clk),
   .mem_access_addr(ALU_out),
   .mem_write_data(reg_read_data_2),
   .mem_write_en(mem_write),
   .mem_read(mem_read),
   .mem_read_data(mem_read_data)
  );
// write back
assign reg_write_data = (mem_to_reg == 1'b1)?  mem_read_data: ALU_out;
// output to control unit
assign opcode = instr[15:12];
endmodule
```

C:/Users/kabir/Vivado/RISCprocessor/RISCprocessor.srcs/sources_1/new/Control_Unit.v

```
`timescale 1ns / 1ps
module Control_Unit(
      input[3:0] opcode,
      output reg[1:0] alu_op,
      output reg jump,beq,bne,mem_read,mem_write,alu_sr
    );
always @(*)
begin
 case(opcode)
 4'b0000:  // LW
   begin
    reg_dst = 1'b0;
    alu_src = 1'b1;
    mem_to_reg = 1'b1;
    reg_write = 1'b1;
    mem_read = 1'b1;
    mem_write = 1'b0;
    beq = 1'b0;
    bne = 1'b0;
    alu_op = 2'b10;
    jump = 1'b0;
   end
 4'b0001:  // SW
   begin
    reg_dst = 1'b0;
    alu_src = 1'b1;
    mem_to_reg = 1'b0;
    reg_write = 1'b0;
    mem_read = 1'b0;
    mem_write = 1'b1;
    beq = 1'b0;
    bne = 1'b0;
    alu_op = 2'b10;
    jump = 1'b0;
   end
 4'b0010:  // data_processing
   begin
    reg_dst = 1'b1;
    alu_src = 1'b0;
    mem_to_reg = 1'b0;
    reg_write = 1'b1;
    mem_read = 1'b0;
    mem_write = 1'b0;
    beq = 1'b0;
```

Verilog code for data path of the processor

C:/Users/kabir/Vivado/RISCprocessor/RISCprocessor.srcs/sources_1/new/Control_Unit.v

```
45          bne = 1'b0;
46          alu_op = 2'b00;
47          jump = 1'b0;
48        end
49      4'b0011:  // data_processing
50        begin
51          reg_dst = 1'b1;
52          alu_src = 1'b0;
53          mem_to_reg = 1'b0;
54          reg_write = 1'b1;
55          mem_read = 1'b0;
56          mem_write = 1'b0;
57          beq = 1'b0;
58          bne = 1'b0;
59          alu_op = 2'b00;
60          jump = 1'b0;
61        end
62      4'b0100:  // data_processing
63        begin
64          reg_dst = 1'b1;
65          alu_src = 1'b0;
66          mem_to_reg = 1'b0;
67          reg_write = 1'b1;
68          mem_read = 1'b0;
69          mem_write = 1'b0;
70          beq = 1'b0;
71          bne = 1'b0;
72          alu_op = 2'b00;
73          jump = 1'b0;
74        end
75      4'b0101:  // data_processing
76        begin
77          reg_dst = 1'b1;
78          alu_src = 1'b0;
79          mem_to_reg = 1'b0;
80          reg_write = 1'b1;
81          mem_read = 1'b0;
82          mem_write = 1'b0;
83          beq = 1'b0;
84          bne = 1'b0;
85          alu_op = 2'b00;
86          jump = 1'b0;
87        end
88      4'b0110:  // data_processing
```

C:/Users/kabir/Vivado/RISCprocessor/RISCprocessor.srcs/sources_1/new/Control_Unit.v

```
89        begin
90          reg_dst = 1'b1;
91          alu_src = 1'b0;
92          mem_to_reg = 1'b0;
93          reg_write = 1'b1;
94          mem_read = 1'b0;
95          mem_write = 1'b0;
96          beq = 1'b0;
97          bne = 1'b0;
98          alu_op = 2'b00;
99          jump = 1'b0;
100       end
101     4'b0111:  // data_processing
102       begin
103         reg_dst = 1'b1;
104         alu_src = 1'b0;
105         mem_to_reg = 1'b0;
106         reg_write = 1'b1;
107         mem_read = 1'b0;
108         mem_write = 1'b0;
109         beq = 1'b0;
110         bne = 1'b0;
111         alu_op = 2'b00;
112         jump = 1'b0;
113       end
114     4'b1000:  // data_processing
115       begin
116         reg_dst = 1'b1;
117         alu_src = 1'b0;
118         mem_to_reg = 1'b0;
119         reg_write = 1'b1;
120         mem_read = 1'b0;
121         mem_write = 1'b0;
122         beq = 1'b0;
123         bne = 1'b0;
124         alu_op = 2'b00;
125         jump = 1'b0;
126       end
127     4'b1001:  // data_processing
128       begin
129         reg_dst = 1'b1;
130         alu_src = 1'b0;
131         mem_to_reg = 1'b0;
132         reg_write = 1'b1;
```

C:/Users/kabir/Vivado/RISCprocessor/RISCprocessor.srcs/sources_1/new/Control_Unit.v

```
      mem_read = 1'b0;
      mem_write = 1'b0;
      beq = 1'b0;
      bne = 1'b0;
      alu_op = 2'b00;
      jump = 1'b0;
    end
  4'b1011:  // BEQ
    begin
      reg_dst = 1'b0;
      alu_src = 1'b0;
      mem_to_reg = 1'b0;
      reg_write = 1'b0;
      mem_read = 1'b0;
      mem_write = 1'b0;
      beq = 1'b1;
      bne = 1'b0;
      alu_op = 2'b01;
      jump = 1'b0;
    end
  4'b1100:  // BNE
    begin
      reg_dst = 1'b0;
      alu_src = 1'b0;
      mem_to_reg = 1'b0;
      reg_write = 1'b0;
      mem_read = 1'b0;
      mem_write = 1'b0;
      beq = 1'b0;
      bne = 1'b1;
      alu_op = 2'b01;
      jump = 1'b0;
    end
  4'b1101:  // J
    begin
      reg_dst = 1'b0;
      alu_src = 1'b0;
      mem_to_reg = 1'b0;
      reg_write = 1'b0;
      mem_read = 1'b0;
      mem_write = 1'b0;
      beq = 1'b0;
      bne = 1'b0;
      alu_op = 2'b00;
      alu_op = 2'b00;
      jump = 1'b1;
    end
  default: begin
      reg_dst = 1'b1;
      alu_src = 1'b0;
      mem_to_reg = 1'b0;
      reg_write = 1'b1;
      mem_read = 1'b0;
      mem_write = 1'b0;
      beq = 1'b0;
      bne = 1'b0;
      alu_op = 2'b00;
      jump = 1'b0;
    end
  endcase
  end
endmodule
```

Verilog code for Control Unit

C:/Users/kabir/Vivado/RISCprocessor/RISCprocessor.srcs/sources_

```
`timescale 1ns / 1ps
module Risc_16_bit(
 input clk
);
 wire jump,bne,beq,mem_read,mem_write,alu_
 wire[1:0] alu_op;
 wire [3:0] opcode;
 // Datapath
 Datapath_Unit DU
 (
  .clk(clk),
  .jump(jump),
  .beq(beq),
  .mem_read(mem_read),
  .mem_write(mem_write),
  .alu_src(alu_src),
  .reg_dst(reg_dst),
  .mem_to_reg(mem_to_reg),
  .reg_write(reg_write),
  .bne(bne),
  .alu_op(alu_op),
  .opcode(opcode)
 );
 // control unit
 Control_Unit control
 (
  .opcode(opcode),
  .reg_dst(reg_dst),
  .mem_to_reg(mem_to_reg),
  .alu_op(alu_op),
  .jump(jump),
  .bne(bne),
  .beq(beq),
  .mem_read(mem_read),
  .mem_write(mem_write),
  .alu_src(alu_src),
  .reg_write(reg_write)
 );

endmodule
```

Verilog code for 16 bit RISC processor

C:/Users/Komal/Downloads/RISCprocessor-20190423T091317Z-001/RISC

```
`timescale 1ns / 1ps
`include "Parameter.v"
// Verilog code for RISC Processor
// Verilog testbench code to test the proc
module test_Risc_16_bit;
 // Inputs
 reg clk;
 // Instantiate the Unit Under Test (UUT)
 Risc_16_bit uut (
  .clk(clk)
 );
 initial
  begin
   clk <=0;
   `simulation_time;
   $finish;
  end
 always
  begin
   #5 clk = ~clk;
  end
endmodule
```

Verilog code for testbench to test the processor

```
C:/Users/kabir/Vivado/RISCprocessor/RISCprocessor.srcs/sim_1/new/Para
1  `timescale 1ns / 1ps
2
3  `ifndef PARAMETER_H_
4  `define PARAMETER_H_
5
6  // Verilog code for RISC Processor
7  // Parameter file
8  `define col 16 // 16 bits instruction memory, d
9  `define row_i 15 // instruction memory, instruc
10 `define row_d 8 // The number of data in data m
11 `define filename "./test/50001111_50001212.o"
12 `define simulation_time #160
13
14 `endif
```

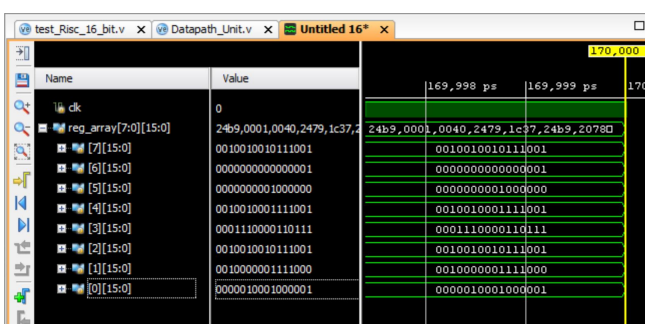
Verilog code for RISC processor Parameter file

D. Simulation

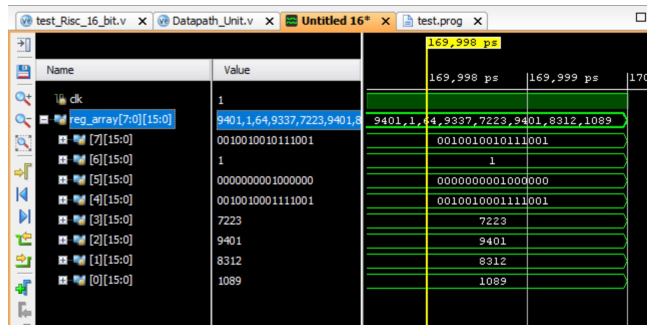
For the simulation, the following operations have been performed and results have been stored in registers ranging from R0-R7. R0 and R1 are used for simply loading data into it. R2 stores the result of the **ADD** operation on values contained in R0 and R1. Other such operations are **Subtraction, Bitwise AND, Bitwise OR, Set Less Than, Hamming distance (XOR)** and a **Jump** operation.

```
C:/Users/kabir/Vivado/RISCprocessor/RISCprocessor.srcs/sim_1/new/test.prog
1 0000_0100_0000_0000 // load R0 <- Mem(R2 + 0)
2 0000_0100_0100_0001 // load R1 <- Mem(R2 + 1)
3 0010_0000_0101_0000 // Add R2 <- R0 + R1
4 0011_0010_0001_1000 // Subtract R3 <- R1 - R0
5 1000_0000_0110_0000 // R4 <- R0 & R1
6 0111_0000_0110_1000 // R5 <- R0 & R1
7 1001_0000_0111_0000 // SLT R6, R0, R1
8 1010_0000_0111_1000 // Hamming Distance between R0 and R1
9 1101_0000_0000_0000 // Jump to the beginning address
```

Instruction memory file used for simulation



Simulation Results (Binary)



Simulation Results (Decimal)

VIII. Conclusion

Both are suitable at their own specific applications. Today, we use both RISC and CISC technology, and we call it **E.P.I.C** (Explicitly Parallel Instruction Computing)

Now we think of whether the RISC v CISC debate is still applicable today. It certainly was applicable in the 1980s and 90s when RISC came into the picture. This is what **David Patterson** had to say about it:

> "Even a million transistors does not go far if a whole computer has to be built from it. That raises the question as to whether the extra hardware needed to implement CISC is the best way to use this 'scarce' resource"

Of course, that was a million transistors, that was back then. Today, even an SoC (System on a chip) has got 6.9 BILLION transistors in it. Back then something that didn't even give much of a result in terms of performance, like a branch predictor, was thought of as a big deal. Nowadays, almost all processors use efficient branch predictors that can predict if there's a branch coming up in the next 10 to 20 instructions. But here's an interesting fact that most people don't know. Starting with the Pentium Pro, Intel actually changed the way they designed their processors and this idea of a thing called Micro opt. What it does is that when it sees an instruction that says add one to this address in memory, it breaks it down into three separate instructions and sends those down the pipeline. So that means that today, a lot of CISC computers actually break down complicated commands in to RISC instructions. Intel added some extra logic at the beginning to split the variable length instructions into fixed length using brute force. That's why x86 hasn't been popular in smartphones, because the extra logic at the beginning actually produces extra heat and uses up more battery and is hence just not as power efficient as the ARM counterparts.

From our simulation, we observe that it was fairly easy to simulate a RISC processor as it has relatively simpler instruction formats and addressing modes. The same task would've been a much tougher ask had we tried to simulate a CISC processor due to the increased complexity of instructions and variable instruction size.

ACKNOWLEDGMENT

Kabir Kanha Arora and Komal Talwar would like to thank their Computer Architecture teacher at Shiv Nadar University, **Dolly Sharma**, for the opportunity to engage in this endeavour. It was a fantastic learning experience and they are grateful for her guidance and constant support. They would also like to thank **Shaik Jani Babu**, a PhD student at Shiv Nadar University for sharing his knowledge of Verilog with them which proved to be very useful.

REFERENCES

[1] Patterson, David A., and Carlo H. Sequin. "RISC I: A reduced instruction set VLSI computer." In Proceedings of the 8th annual symposium on Computer Architecture, pp. 443-457. IEEE Computer Society Press, 1981.

[2] Cocke, John, and Victoria Markstein. "The evolution of RISC technology at IBM." IBM Journal of research and development 34.1 (1990): 4-11.

[3] Hennessy, John L., and David A. Patterson. Computer architecture: a quantitative approach. Elsevier, 2011.

[4] Mano, M. Morris. Digital design. EBSCO Publishing, Inc., 2002.

[5] Blem, Emily, Jaikrishnan Menon, and Karthikeyan Sankaralingam. "Power struggles: Revisiting the RISC vs. CISC debate on contemporary ARM and x86 architectures." 2013 IEEE 19th International Symposium on High Performance Computer Architecture (HPCA). IEEE, 2013.

[6] Jamil, Tariq. "Risc versus cisc." Ieee Potentials 14.3 (1995): 13-16.

[7] Akram, Ayaz. "A Study on the Impact of Instruction Set Architectures on Processor's Performance." (2017).